

Diseño, Correctitud y Análisis de Complejidad de un Range Tree 2D con Fractional Cascading

Autores: Thiago Rivera, Wilder Bañon, Tamy Flores, Jesús Niño

Proyecto 2: Efecto Cascada

CS3014 - Estructuras de Datos Avanzadas

Universidad de Ingeniería y Tecnología (UTEC) - Lima, Perú

Resumen—En este informe se presenta el diseño, análisis matemático de correctitud y evaluación de complejidad de un *Layered Range Tree* bidimensional optimizado con *Fractional Cascading*. Esta estructura resuelve consultas de conteo ortogonal sobre un conjunto de hasta $3 \cdot 10^5$ puntos en tiempo $O(\log n)$, superando el límite teórico de $O(\log^2 n)$ de los árboles de rangos clásicos. Se formaliza el invariante de los arreglos en cascada, la lógica de preprocesamiento ascendente tipo *MergeSort* y el descenso directo en $O(1)$. Asimismo, se demuestra mediante inducción y recurrencias matemáticas que el uso de memoria y tiempo de construcción se mantienen estrictamente en $O(n \log n)$.

Palabras clave—Range Tree, Fractional Cascading, Geometría Computacional, Búsqueda Ortogonal.

1. INTRODUCCIÓN

El problema de búsqueda en rangos ortogonales exige identificar elementos multidimensionales contenidos en hiper-rectángulos de consulta [2]. En su versión bidimensional de conteo, dado un conjunto de puntos P y una caja $R = [x_1, x_2] \times [y_1, y_2]$, se debe retornar $|P \cap R|$ [1].

La solución clásica emplea un *Range Tree 2D*, el cual requiere $O(\log^2 n)$ por consulta debido a búsquedas binarias redundantes en árboles secundarios [2]. Dadas las restricciones de $n, q \leq 3 \cdot 10^5$ y un límite de 5 segundos [1], este proyecto implementa un *Layered Range Tree* aplicando *Fractional Cascading* (o *cross-linking*). Esta técnica permite unificar las búsquedas secundarias precalculando punteros fraccionarios, reduciendo el tiempo de consulta a $O(\log n)$ [2].

2. ARQUITECTURA Y DISEÑO DE LA ESTRUCTURA

2.1. Subconjuntos Canónicos y Arreglos Secundarios

Sea T el árbol principal construido sobre la dimensión X . Cada nodo $u \in T$ define un subconjunto canónico $P(u)$ que contiene las hojas de su subárbol [2]. En lugar de usar un BST secundario, en un *Layered Range Tree*, $P(u)$ se almacena en un arreglo contiguo currentY , el cual está estrictamente ordenado respecto a la coordenada Y [1, 2]. Para minimizar el uso de memoria, solo la raíz de T preserva explícitamente las coordenadas Y de los puntos en rootY .

2.2. Puentes Estructurales (Fractional Cascading)

La clave del diseño radica en la conexión de las estructuras secundarias. Cada nodo u mantiene dos arreglos de enteros, toLeft y toRight , mapeados uno a uno con su arreglo currentY [1].

Definición 1 (Puentes de Cascada [2]). *Para un elemento con índice i en el arreglo currentY del nodo u , el valor $\text{toLeft}[u][i]$ almacena el menor índice j en el hijo izquierdo tal que $\text{leftY}[j] \geq \text{currentY}[i]$. Análogamente se define $\text{toRight}[u][i]$ para el hijo derecho.*

Esta arquitectura desacopla el costo de la búsqueda; en lugar de hacer repetidos *binary searches*, un índice en el nodo u se traduce a su índice correspondiente en los nodos hijos en tiempo $O(1)$.

3. MECÁNICA DE OPERACIONES

3.1. Preprocesamiento Ascendente (*build*)

La construcción de los arreglos se ejecuta *bottom-up*. En las hojas, currentY contiene un único elemento. En los nodos internos, se combinan los arreglos leftY y rightY de sus hijos mediante una técnica *Two-Pointers* idéntica a la fase de mezcla de *MergeSort* [1].

Inmediatamente después de la fusión, se computan los puentes toLeft y toRight . Se itera sobre el arreglo recién formado currentY con un puntero i , y se avanzan los punteros locales j y k en leftY y rightY respectivamente para registrar las transiciones.

3.2. Consulta de Conteo (*query*)

La consulta evalúa el rectángulo $R = [x_1, x_2] \times [y_1, y_2]$ de la siguiente forma [1]:

1. Se localizan los límites $[ql, qr]$ en el eje X sobre el arreglo de puntos base.
2. Se ejecuta la **única** búsqueda binaria sobre el eje Y en el arreglo rootY de la raíz, obteniendo los índices $[y_{low}, y_{high}]$. Matemáticamente, esto se traduce en computar el *lower bound* (límite inferior estricto) para y_1 y el *upper bound* (límite superior estricto) para y_2 , delimitando con precisión el intervalo de interés.
3. Se inicia un recorrido descendente desde la raíz. Si el rango horizontal del nodo u está completamente contenido en $[ql, qr]$, se suma $(y_{high} - y_{low})$ al total y no se desciende más.
4. Si se debe descender, los nuevos límites para los hijos se obtienen mediante $y'_{low} = \text{toLeft}[u][y_{low}]$ y $y'_{high} = \text{toLeft}[u][y_{high}]$, realizando la transición al hijo izquierdo en tiempo $O(1)$. Lo mismo aplica al hijo derecho con toRight .

4. ANÁLISIS DE CORRECTITUD

Teorema 1 (Correctitud de Transición Fraccionaria). *Si el intervalo de índices $[y_{low}, y_{high}]$ en el nodo u contiene exactamente los puntos de $P(u)$ cuyas coordenadas Y están en $[y_1, y_2]$, entonces el intervalo $[\text{toLeft}[u][y_{low}], \text{toLeft}[u][y_{high}]]$ delimita exactamente los puntos correspondientes en el hijo izquierdo.*

Proof. Procedemos por inducción sobre la profundidad del descenso en el árbol. **Caso base:** En la raíz, la búsqueda binaria inicial garantiza que el intervalo $[y_{low}, y_{high}]$ acota de manera exacta a los elementos cuyas coordenadas $Y \in [y_1, y_2]$. **Hipótesis inductiva:** Supongamos que en un nodo u , el intervalo de índices $[y_{low}, y_{high}]$ delimita exactamente al subconjunto $P(u) \cap R_Y$, donde $R_Y = [y_1, y_2]$. **Paso inductivo:** Sea v el hijo izquierdo de u . Al aplicar $y'_{low} = \text{toLeft}[u][y_{low}]$, por la Definición 1, y'_{low} es el menor índice en v tal que $\text{leftY}[y'_{low}] \geq \text{currentY}[y_{low}] \geq y_1$. Simétricamente, $y'_{high} = \text{toLeft}[u][y_{high}]$ mapea al primer elemento que excede estrictamente y_2 . Dado que $P(v) \subseteq P(u)$ y ambos arreglos mantienen un ordenamiento estricto, la correspondencia biunívoca preserva las cotas. Por lo tanto, el nuevo intervalo $[y'_{low}, y'_{high}]$ delimita exactamente a $P(v) \cap R_Y$. $\square$

Lema 1 (Exactitud del Conteo Canónico). *Si un nodo u es canónico respecto al rango de la consulta en X (es decir, $X_u \subseteq [x_1, x_2]$), entonces $y_{high} - y_{low}$ es exactamente el número de puntos en $P(u) \cap R$.*

Proof. Al ser u canónico en X , todos los puntos en $P(u)$ satisfacen la restricción horizontal. Como el intervalo semiabierto $[y_{low}, y_{high})$ abarca todos los puntos en $P(u)$ que caen en $[y_1, y_2]$ (demostrado por el Teorema 1), y la estructura utiliza indexación desde cero, la diferencia aritmética simple $y_{high} - y_{low}$ determina con exactitud la cardinalidad del conjunto sin necesidad de iteración adicional ni correcciones de desplazamiento (offset). $\square$

5. ANÁLISIS FORMAL DE COMPLEJIDAD

5.1. Complejidad Espacial

En un árbol perfectamente balanceado, cada punto original aparece exactamente en un nodo por cada nivel de profundidad. Un nodo a profundidad d mantiene su arreglo canónico, cuyos arreglos de puentes `toLeft` y `toRight` son de tamaño $|P(u)| + 1$. Sea $S(n)$ el espacio total, la suma de los tamaños de los arreglos en cualquier nivel fijo es $O(n)$ [2]. Como la altura del árbol base es $\log_2 n$, tenemos:

$$S(n) = \sum_{d=0}^{\log_2 n} O(n) = O(n \log n) \quad (1)$$

5.2. Tiempo de Procesamiento

La construcción recursiva genera la recurrencia $T(n) = 2T(n/2) + f(n)$. El término $f(n)$ corresponde a la mezcla (Merge) y el cálculo de puentes. Como ambos procedimientos utilizan punteros que iteran secuencialmente sin retroceso, procesar un nodo toma tiempo estrictamente proporcional a su tamaño: $f(n) = O(n)$ [1]. Evaluando la recurrencia mediante el Teorema Maestro:

$$T(n) = 2T(n/2) + O(n) \implies T(n) = O(n \log n) \quad (2)$$

5.3. Tiempo de Consulta en el Peor Caso

El tiempo de consulta $Q(n)$ depende de la búsqueda inicial y el recorrido del árbol.

- **Fase 1:** $O(\log n)$ para hallar los límites iniciales en X y en Y .
- **Fase 2:** En el peor caso, el rango X se descompone en a lo sumo $2 \log_2 n$ subárboles canónicos (nodos delimitadores).
- **Fase 3:** Por Fractional Cascading, evaluar la condición y bifurcar los límites toma tiempo constante $O(1)$ por nodo visitado.

Por ende, el tiempo de consulta total es:

$$Q(n) = \underbrace{O(\log n)}_{\text{Búsquedas base}} + \underbrace{O(\log n) \times O(1)}_{\text{Descenso con punteros}} = O(\log n) \quad (3)$$

Al ejecutar q consultas, el tiempo global del algoritmo resulta $O(n \log n + q \log n)$, superando ampliamente el margen requerido por el problema ($O(q \log^2 n)$).

6. CONCLUSIONES

1. Se diseñó e implementó un *Layered Range Tree 2D*, integrando *Fractional Cascading* mediante puentes estáticos calculados *bottom-up*.
2. El análisis demostró que el cálculo exhaustivo de índices fraccionarios no altera la cota teórica de preprocesamiento, la cual se mantiene en $O(n \log n)$.
3. La sustitución de operaciones repetidas de búsqueda binaria por simples traslaciones de índices acotó estrictamente la consulta de recuento ortogonal a $O(\log n)$.
4. La eliminación de estructuras asociadas complejas a favor de arreglos paralelos optimizó la localidad en caché, garantizando un desempeño robusto dentro de los límites de tiempo del sistema de evaluación.

REFERENCES

- [1] UTEC - Ciencia de la Computación, *Documentos del curso: Proyecto 2 - Efecto Cascada (Instrucciones y código base)*, 2026.
- [2] M. de Berg, O. Cheong, M. van Kreveld, y M. Overmars, *Computational Geometry: Algorithms and Applications*, 3ra ed. Capítulo 5: Orthogonal Range Searching.